qae_european: how to use the kit

Status: version 1.0.0. Independent review done and its findings addressed (2026-11-10, `docs/review/`). Awaiting sign-off.

Prices a European call under Black-Scholes with **maximum-likelihood amplitude estimation** (MLAE) on a quantum circuit simulator, and compares it with Black-Scholes, Monte Carlo and quasi-Monte Carlo.

It is the simplest circuit we could build that still exercises the full amplitude-estimation method. A European call has a closed form, so this kit is not a case for using a quantum computer to price one: it is a test bench for the method, its benchmarks and its validation.

Document	Content	Status
This guide	Install, run, reproduce, extend	Current
Document 1: Circuit	The circuit, environment, inputs, outputs, reproducibility	Written
Document 2: Benchmarks	The comparators, why they were chosen, their code and parameters	Written
Document 3: Tests	Pre-registered test plan (Part 1), results and verdicts (Part 2)	Written
Independent review	The reviewer's report and the developer's response, finding by finding	Done

The documents are also exported to Word, LaTeX and PDF in `docs/export/` To rebuild them: `python docs/build_benchmarks_doc.py` in the kit folder, then `python tools/export/export_docs.py kits/qae_european` from the repository root. This needs pandoc, XeLaTeX (TeX Live with fontspec and sectsty) and the DejaVu fonts.

Earlier exploratory documents are kept in `docs/archive/` for the record. They describe a previous version of the code and are superseded.

---

1. Install

Python 3.10 or later (tested on 3.13).

To reproduce the published results exactly, use the frozen environment:

```
git clone https://github.com/romaindutrenois-gif/veqctor-quantum.git
cd veqctor-quantum/kits/qae_european
python -m venv .venv && source .venv/bin/activate
pip install -r requirements-lock.txt
pip install --no-deps -e .
```

To use the kit in your own environment (versions within the ranges in `pyproject.toml`):

```
pip install -e ".[test]"
```

Check the install:

```
pytest tests          # 41 test cases (21 test functions), a few seconds
```

No account or quantum hardware is needed: everything runs on a local simulator.

2. Price one option

```
qae-european price --spot 100 --strike 105 --rate 0.03 --vol 0.2 --maturity 1 --qubits 5 --depth 8
 ↪  --seed 1
```

Output in the frozen environment (rounded here to 5 decimals; the command prints full precision):

```
{
  "black_scholes": 7.12806,
  "grid_price": 7.11203,
  "circuit_exact": 7.11203,
  "amplitude_estimation": 7.09878,
  "calls": 26200,
  "monte_carlo": {"value": 6.97917, "calls": 26200, "stderr": 0.07683},
  "quasi_monte_carlo": {"value": 7.12759, "calls": 16384, "stderr": null}
}
```


How to read it:

- `black_scholes` is the reference price.
- `grid_price` is the price on the discretised distribution (`2^5` = 32 points). The gap to Black-Scholes is the **discretisation bias**. It shrinks as `--qubits` grows, but only down to a floor set by the truncation of the grid at ±4 standard deviations: about −0.05% for this option, up to −2% for a 5-year option at 80% volatility (Document 1, section 3).
- `circuit_exact` is the price read from the exact state of the circuit. It must equal `grid_price` up to numerical precision (a relative 10^{-12} or better): this checks the circuit.
- `amplitude_estimation` is one MLAE estimate using `calls` applications of the circuit A. Its error against `grid_price` is the **statistical error**.
- `monte_carlo` uses the same number of calls (paths). Quasi-Monte Carlo uses the largest power of two not above it (a Sobol requirement). Its `stderr` is not defined for a single scrambled sequence, hence `null`.

Options: `--qubits` (price qubits, 3 to 7 in the campaign; the circuit has `qubits + 1` qubits) and `--depth` (MLAE ladder: Grover powers 0, 1, 2, 4, ..., `2^(depth−2)`, 100 measurements each).

3. Use it from Python

```python
import numpy as np
from qae_european import Option, LognormalGrid, QuantumPricer, MaximumLikelihoodAE, Ladder,
 ↪  BlackScholes

option = Option(spot=100, strike=105, rate=0.03, vol=0.2, maturity=1.0)
grid = LognormalGrid(option, n_qubits=5)        # lognormal on 32 points, ±4 standard deviations
pricer = QuantumPricer(grid)                    # builds the circuits A and Q

print(pricer.circuit_a().draw())                # the circuit itself (Qiskit)
print(BlackScholes.call(option), grid.price(), pricer.price_exact())

ladder = Ladder.exponential(depth=8, shots=100)
mlae = MaximumLikelihoodAE(ladder, backend="statevector")   # simulates A Q^k in full
print(mlae.estimate_prices(pricer, np.random.default_rng(1), draws=5), ladder.calls)
```

Expected output (frozen environment), after a few seconds:

```
7.1280646692617395 7.1120286452832895 7.112028645282543
[7.09877631 7.14967104 7.09960548 7.12204347 7.11001134] 26200
```

The first estimate is the same as the `amplitude_estimation` value of section 2: same seed, and the two backends give the same probabilities.

Two backends give the same measurement probabilities on a noise-free simulator:

- `"statevector"` simulates every circuit A Q^k with Qiskit. Exact, slow for large powers.
- `"born"` uses the identity $P(1 \text{ after } k \text{ steps}) = \sin^2((2k+1)\theta)$, with θ from the exact state of A. Fast; used for the campaign. The identity is checked by `qae-european verify` and by the unit tests.

4. Run the test campaign

```
qae-european verify                          # H0-b: full simulation of A Q^k on 20 random scenarios,
 ↪  about 15 minutes
qae-european campaign --processes 2          # 1,848 scenarios; measured: about 40 minutes on 2 cores
```

Outputs go to `results/verification.json`, `results/campaign.csv` (one row per scenario, number of qubits and budget; columns described at the top of `src/qae_european/campaign.py`) and `results/run_info.json` (the commit, parameters and library versions of the run). The published results are in the repository.

To check them against the criteria of Document 3 and redraw its figures (needs the docs extra, `pip install -e ".[docs]"`):

```
python docs/analyse.py                       # writes results/summary.json and docs/img/*.png
python docs/checks.py                       # results/checks.json: loader accuracy, MLAE global maximum
python docs/h2a_draws.py                     # results/h2a_draws.json: H2-a with 2,000 draws (about 10
 ↪  minutes)
python docs/truncation.py                    # results/truncation.json: truncation bias floor
python docs/resource_estimate.py             # results/resources.json: resources at a target accuracy
```

The campaign is **resumable**: scenarios already in the CSV are skipped. To split it, use `--start` and `--stop` (scenario indices, 0 to 1,848).

5. Reproduce a single row

Every random stream is derived from a fixed seed (20261103) and the indices of the scenario, qubits, budget and method. Any row can be recomputed on its own:

```
from qae_european.campaign import scenarios, run_scenario

sc = scenarios()[1000]                       # moneyness 1.0, maturity 5Y, vol 60%, rate 0%
row = run_scenario(sc, qubits=(5,), depths=(8,))[0]
print(row)
```

The values must match the row of `results/campaign.csv` with the same `scenario`, `qubits` and `depth`, exactly, in the frozen environment.

6. Code layout

```
src/qae_european/
  option.py              Option: contract and market inputs, with validation
  grid.py               LognormalGrid: discretisation on 2^n points (conditional-
mean prices)
  quantum.py         QuantumPricer: circuit A, Grover operator Q, exact probabilities
  estimation.py           Ladder, MaximumLikelihoodAE
  benchmarks/         BlackScholes, MonteCarlo (continuous or on the grid), QuasiMonteCarlo (scr
  resources.py            circuit size after compilation, power-law fits
  campaign.py             the scenario grid, the runner, the verification
  cli.py                  the qae-european command
tests/test_package.py     unit tests
docs/               Documents 1 to 3, figures (make_figures.py), analysis (analyse.py), exports
results/             the published campaign (campaign.csv, verification.json, run_info.json) and
                   of the analysis scripts (summary, checks, h2a_draws, truncation, resources)
```

7. Adapt it to another payoff

The circuit loads a distribution on n qubits, then rotates an objective qubit by the normalised payoff at each grid point. To price another payoff on the same underlying, change the payoff:

1. Subclass `Option` and override `payoff`. Everything else (`LognormalGrid`, `QuantumPricer`, `MonteCarlo`, `QuasiMonteCarlo`) calls `option.payoff`, so it follows. For a put:

```python
from dataclasses import dataclass
import numpy as np
from qae_european import Option, LognormalGrid, QuantumPricer, MonteCarlo

@dataclass(frozen=True)
class Put(Option):
    def payoff(self, prices):
        return np.maximum(self.strike - np.asarray(prices, dtype=float), 0.0)

put = Put(spot=100, strike=95, rate=0.03, vol=0.2, maturity=1.0)
print(QuantumPricer(LognormalGrid(put, 5)).price_exact())                    # 4.3594 on the grid
print(MonteCarlo().price(put, 200_000, np.random.default_rng(0)).value)      # 4.3797;
 ↪  Black-Scholes put: 4.3720
```

2. `QuantumPricer` normalises the grid payoffs by their maximum and sets one rotation per point. It needs no change as long as payoffs are non-negative.

3. `BlackScholes` prices calls only. Add the closed form of the new payoff, if there is one, as a benchmark (subclass `Benchmark`), and the campaign code will run against it once `campaign.run_scenario` calls it.

Path-dependent payoffs (Asian, barrier) need more qubits per time step and are outside this kit.

8. Scope and limits

The kit is a noise-free simulator **test bench** for maximum-likelihood amplitude estimation. It is not a pricing model, and it should not enter a model inventory as one.

Use it for:

- teaching and demonstrating the full amplitude-estimation chain on the simplest payoff;
- measuring calls against plain Monte Carlo on the same grid, within the tested envelope below;
- a reproducible regression harness for later kits: seeded, row-by-row reproducible, with a pre-registered test plan.

Tested envelope: 3 to 7 price qubits, ladder depths 4 to 8, 100 shots per power, a grid width of 4 standard deviations, and the 1,848 scenarios of Document 3. Within it, two quantities drive the results (Document 3, sections 2.5 to 2.7):

Quantity	Where the kit behaves as documented	Outside
Amplitude a = grid price / (discount × largest grid payoff) s = σ√T, the standard deviation of ln S_T	a ≥ 0.03: beats Monte Carlo on the grid in 99.8% of cases s < 0.8: within 1% of Black-Scholes in 99.0% of cases at 7 qubits	a < 0.01: 89.4%, and a heavy tail of large estimation errors s ≥ 1.3: 3.8%, mostly from a truncation bias floor that more qubits cannot remove

Do not use it for:

- valuation, risk, hedging or any production number: a closed form exists, and quasi-Monte Carlo is about 20 times more accurate at the same budget;
- any claim of quantum advantage, run time or hardware feasibility: there is no noise model and no run on hardware, and the resource estimate (Document 1, section 12) assumes a perfect machine;
- small amplitudes (deep out-of-the-money, or long-dated and high-volatility options);
- s = σ√T of 0.8 or more at the default width;
- more than 7 price qubits (untested; the circuit accepts at most 10);
- payoffs that can be negative (rejected), path-dependent payoffs or several underlyings.

Other limits:

- **Exact loading does not scale.** Both the loader and the payoff encoding grow exponentially with the number of qubits (Document 1, section 8).
- **Calls are not run time.** One call of A on a fault-tolerant machine is far slower than one Monte Carlo path on a GPU. The comparison is in number of calls only (Document 3, section 1.1).
- **The error is scaled by the largest payoff.** The price is read as discount × largest payoff × amplitude, so on wide grids a small amplitude error becomes a large price error.

9. What it would take to go further

This kit is a test bench. Three steps separate it from a pricer or a risk engine:

- **A harder problem and scalable loading.** Amplitude estimation matters where quadrature and quasi-Monte Carlo fail: many dimensions and path dependence. Exact loading must then be replaced, for example by loading standard normals and computing the paths in the circuit (Chakrabarti et al., 2021), or by quantum signal processing (Stamatopoulos and Zeng, 2024).
- **A hardened estimator under noise.** A variant with a confidence interval that holds at small amplitude (for example iterative amplitude estimation), tested under a noise model and at small size on hardware.
- **A fault-tolerant resource estimate.** Logical qubits, T gates and run time at a stated accuracy, against GPU Monte Carlo with variance reduction. The most recent published estimate is about 4,700 logical qubits, 10^9 T gates and a 45 MHz logical clock rate (Stamatopoulos and Zeng, 2024), far beyond current machines. This kit gives only a noise-free count of calls, Grover powers and CNOTs at a target accuracy (Document 1, section 12).

A risk engine also needs sensitivities and many revaluations a day, not one price. Greeks and the cost of repeated runs belong to the second and third steps.

This section adopts the wording proposed by the independent review (`docs/review/`).

References

- Suzuki et al., *Amplitude estimation without phase estimation*, Quantum Information Processing, 2020. arXiv:1904.10246
- Stamatopoulos et al., *Option pricing using quantum computers*, Quantum, 2020. arXiv:1905.02666
- Chakrabarti et al., *A threshold for quantum advantage in derivative pricing*, Quantum, 2021. arXiv:2012.03819
- Stamatopoulos and Zeng, *Derivative pricing using quantum signal processing*, Quantum, 2024. arXiv:2307.14310